Super učni vodič APS2

Super učni vodič APS2

To je glavni “lock-in” dokument. Beri ga aktivno: pri vsakem algoritmu si na papir naredi en mini primer.

1. Asimptotika in ocenjevanje časa

Cilj

Znati oceniti, kako hitro raste čas algoritma, in iz tega sklepati, ali je algoritem uporaben za velik n.

Kdaj uporabim

Ko naloga sprašuje po O, Θ, Ω, primerjavi algoritmov ali oceni časa pri danem računalniku.

Postopek

1. Prepoznaj osnovno obliko: zanke, rekurzija, število povezav v grafu, velikost tabele.
2. Ignoriraj konstante in nižje člene.
3. Če moraš oceniti sekunde: uporabi $T(n)=C*f(n)$ in deli s številom operacij/s.

Mini primer

$n = 10^6$, algoritem $O(n^2)$: približno 10^{12} operacij. Pri 10^9 operacij/s je to približno $1000\ s$, torej okrog 17 minut.

Tipične napake

- Zamenjaš zahtevnost algoritma z zahtevnostjo problema.
- Pri polnem grafu pozabiš, da je $m=\Theta(n^2)$.
- Pri $n \log n$ uporabiš občutek kot da je skoraj n; ni isto, ampak je še vedno veliko boljše kot n^2.

Preverjanje

Če se pri velikem n kvadratični algoritem zdi “čisto hiter”, si skoraj gotovo zgrešil faktor.

2. Amortizirana analiza

Cilj

Dokazati, da je zaporedje operacij poceni v povprečju, čeprav je posamezna operacija včasih draga.

Kdaj uporabim

Dinamična tabela, razširjen sklad, dvojiški števec, tabela tabel, strukture s periodičnim preurejanjem.

Postopek

1. Ne glej samo ene operacije.
2. Preštej skupno delo čez n operacij.
3. Deli z n.

Mini primer: dvojiški števec

Bit 0 se spremeni n-krat, bit 1 $n/2$-krat, bit 2 $n/4$-krat ...

Skupaj $< 2n$, zato je `povecaj()` amortizirano $O(1)$.

Tipične napake

- Rečeš “najslabši primer je $O(k)$, zato je tudi amortizirano $O(k)$” - lahko je, ampak pogosto ni tesno.
- Pri potencialu izbereš funkcijo, ki gre v dolg.

Preverjanje

Vprašaj se: “Ali lahko vsak drag dogodek zaračunam nečemu, kar se zgodi omejeno velikokrat?”

3. Tabele, slovarji, zgoščevanje

Zgoščena tabela

Cilj

Hitro vstavljanje, iskanje in brisanje po ključu.

Kdaj uporabim

Ko imamo ključe in želimo pričakovano $O(1)$ dostop.

Postopek

1. Izračunaj $h(k)$.
2. Če je veriženje: vstavi v koš.
3. Če je odprto naslavljanje: išči naslednje mesto z $h(k,i)$.
4. Spremljaj faktor zasedenosti $\alpha=n/m$.

Mini primer

$h(k)=k \bmod 5$, ključ 17: $17 \bmod 5 = 2$, gre v koš 2.

Pazi

- Pri linearnem naslavljanju nastane grupiranje.
- Pri brisanju v odprtem naslavljanju ne smeš samo izprazniti celice; uporabiš `DELETED`.
- Pri duplikatih ključa slovar običajno posodobi vrednost.

Bloomov filter

Cilj

Zelo prostorsko poceni test pripadnosti.

Odgovori

- “Zagotovo NE”
- “Verjetno DA”

Pazi

False positive je možen. False negative ni možen, razen če nepravilno brišeš.

4. Drevesa

BST

Kdaj uporabim

Urejeni ključi, iskanje minimuma/maksimuma, predhodnik/naslednik.

Koraki iskanja

1. Primerjaj s korenom.
2. Manjše gre levo, večje desno.
3. Ponavljaj do zadetka ali praznega otroka.

Zahtevnost

`O(h)`, kjer je h višina. Uravnoteženo: `O(log n)`, degenerirano: `O(n)`.

AVL

Cilj

BST, ki po vsaki spremembi ostane uravnotežen.

Pravilo

Za vsako vozlišče: `|višina(levo)-višina(desno)| <= 1`.

Rotacije

- LL -> desna rotacija.
- RR -> leva rotacija.
- LR -> leva pri levem otroku, nato desna.
- RL -> desna pri desnem otroku, nato leva.

Mini trik

Najprej najdi najnižje neuravnoteženo vozlišče. Tam odloči tip rotacije.

Rdeče-črno drevo

Kdaj uporabim

Ko želiš `O(log n)` in manj strogo uravnoteženje kot AVL.

Pravila

- Koren je črn.
- Rdeč nima rdečega otroka.
- Vse poti do praznih listov imajo enako črno višino.

Pazi

Novo vozlišče vstaviš kot rdeče. Če je starš črn, si končal. Če je starš rdeč, gledaš strica:

- rdeč stric -> prebarvanje,
- črn stric -> rotacija + prebarvanje.

5. Kopice in prioritetna vrsta

Cilj

Hitro dobiti najmanjši/največji element.

Min-kopica

- `min`: `O(1)`
- `insert`: `O(log n)`
- `deleteMin`: `O(log n)`
- `heapify` celotne tabele: `O(n)`

Tipične napake

- Najmanjši element v min-kopici je v korenu, največji je nekje med listi.
- Kopica ni BST. Le lokalni odnos starš-otrok je urejen.

6. Disjunktne množice

Cilj

Hitro preverjati, ali sta elementa v isti komponenti.

Operacije

- `makeSet(x)`

- `find(x)`
- `union(x,y)`

Kdaj uporabim

Kruskal, povezane komponente, združevanje skupin.

Trik

Vedno uporabi:

- union by rank/size,
- path compression.

Potem je amortizirano skoraj `O(1)`.

7. Dinamično programiranje

Cilj

Problem razbiješ na prekrivajoče se podprobleme in rezultate shraniš.

Kako prepoznaš DP

- “Največja/najmanjša vrednost”
- “Na koliko načinov”
- “Optimalna postavitev”
- “Dovoljene odločitve po korakih”

Postopek

1. Definiraj stanje `dp[...]`.
2. Zapiši prehod.
3. Zapiši osnovne primere.
4. Izberi vrstni red računanja.
5. Iz rezultata po potrebi rekonstruiraj izbire.

Stopnice

`f(n)=f(n-1)+f(n-2)`, če so koraki 1 ali 2.

Osnovi: `f(0)=1`, `f(1)=1`.

0/1 nahrbtnik

`dp[i][w]` = najboljša vrednost s prvimi `i` predmeti in kapaciteto `w`.

Prehod:

```
dp[i][w] = max(dp[i-1][w], dp[i-1][w-w_i] + v_i)
```

Če `w_i > w`, predmeta ne moreš vzeti.

Delni nahrbtnik

Če smeš rezati predmete, ni DP. Uporabi požrešno:

1. Izračunaj `v_i/w_i`.
2. Uredi padajoče.
3. Jemlji najboljše razmerje, zadnji predmet po potrebi odreži.

Verižno množenje matrik

`c(i,j)=min(c(i,k)+c(k+1,j)+d_{i-1}d_kd_j)`

Računaš po diagonalah. Rezultat je `c(1,n)`.

8. Grafi - samo zahtevani del

Prim

Kdaj

MST, povezan neusmerjen utežen graf.

Koraki

1. Začni v vozlišču.
2. Množico drevesa širi z najcenejšo povezavo navzven.
3. Ustavi se pri `n-1` povezavah.

Preverjanje

MST ima vedno `n-1` povezav in poveže vsa vozlišča brez cikla.

Kruskal

Koraki

1. Uredi povezave po teži.
2. Dodajaj najcenejše, ki ne naredijo cikla.
3. Cikle preverjaj z disjunktnimi množicami.

Pazi

Če ima graf več komponent, dobiš minimalni vpeti gozd, ne drevesa.

Topološko urejanje

Kdaj

Samo DAG.

Koraki

1. Vhodne stopnje.
2. V vrsto vozlišča z vhodno stopnjo 0.
3. Odstranjuj in zmanjšuj stopnje.
4. Če ostane vozlišče, ima graf cikel.

Dijkstra

Kdaj

Najkrajše poti z nenegativnimi utežmi.

Pazi

Negativna utež -> Dijkstra odpade.

Bellman-Ford

Kdaj

Negativne uteži, zaznava negativnega cikla.

Koraki

1. Inicializiraj razdalje.
2. `n-1` krat sprosti vse povezave.
3. Dodatni prehod zazna negativen cikel.

Welsh-Powell

Kdaj

Hitro barvanje grafa.

Koraki

1. Uredi vozlišča padajoče po stopnji.
2. Vzemi prvo nepobarvano, dodeli novo barvo.
3. Z isto barvo pobarvaj nesosednja vozlišča.
4. Ponavljaj.

Pazi

Ni nujno optimalen.

9. Kompleksnost P/NP

P

Odločitveni problemi, rešljivi v polinomskem času.

NP

Problemi, pri katerih lahko dano rešitev preverimo v polinomskem času.

NP-poln

Problem je v NP in vsak NP problem se nanj polinomsko reducira.

NP-težak

Vsaj tako težak kot NP-polni problemi, ni nujno v NP.

Tipična izpitna past

Če ima problem znan eksponenten algoritem, to ne pomeni, da problem ni v P. Pomeni samo, da s tem algoritmom pokažeš zgornjo mejo.

10. Najbolj tipične izpitne naloge

“DA/NE in utemelji”

Piši kratko:

1. DA/NE.
2. En stavek definicije.
3. Protiprimer, če je NE.

“Izvedi algoritem na grafu”

Vedno pokaži tabelo korakov:

- trenutna množica,
- kandidati,
- izbrana povezava/vozlišče,
- posodobljene razdalje/barve.

“DP naloga”

Ne začni s kodo. Najprej stanje in rekurenca. Brez tega rešitev izgleda kot ugibanje.

“Drevo po vstavljanju”

Po vsakem vstavljanju preveri najnižji neuravnotežen prednik. Ne popravljaj od korena navzdol.